

hwk2/tb2_arithmetic.vhd

```

1 -----
2 -- TB2_Arithmetic -- SH-2 CPU Integration Testbench 2
3 --
4 -- Two phases controlled by a single signal "loading":
5 --   loading='1' : TB owns the bus (load hex, then check memory)
6 --               CPU is held in reset so it tri-states all its outputs
7 --   loading='0' : CPU owns the bus (program execution)
8 --               TB drives 'Z' so it doesn't interfere
9 --
10 -- CPU, memory, and TB all share the same resolved bus signals.
11 -- Only one non-Z driver is active at any given time.
12 --
13 -- Instructions covered:
14 --   ADD Rm,Rn    ADD #imm,Rn    SUB Rm,Rn
15 --   ADDC (two cases: no carry-out, carry-out)
16 --   ADDV (overflow)  NEG  NEGC
17 --   DT (decrement-and-test loop x3)
18 --   CMP/EQ CMP/HS CMP/GT CMP/PZ  SUBV (underflow)
19 --   MOVT  CLRT
20 --
21 -- Expected results in data memory (block 1):
22 --   0x1000  0x00000008  ADD 5+3=8
23 --   0x1004  0x00000002  ADD #-3 -> 2
24 --   0x1008  0x00000007  SUB 10-3=7
25 --   0x100C  0xFFFFFFFF  ADDC no carry: 0xFFFFFFFF+1+T=0
26 --   0x1010  0x00000000  ADDC carry: result=0
27 --   0x1014  0x00000001  ADDC carry: T=1 stored
28 --   0x1018  0x80000000  ADDV overflow result
29 --   0x101C  0xFFFFFFF8  NEG 5
30 --   0x1020  0xFFFFFFF8  NEG 0-1=0
31 --   0x1024  0x00000001  DT: T=1 when count=0
32 --   0x1028  0x00000001  CMP/EQ 42==42: T=1
33 --   0x102C  0x00000001  CMP/HS 10>=5 unsigned: T=1
34 --   0x1030  0x00000001  CMP/GT 6>5 signed: T=1
35 --   0x1034  0x00000001  CMP/PZ 0>=0: T=1
36 --   0x1038  0x7FFFFFFF  SUBV 0x80000000-1 underflow
37 --
38 -- Program image (assembled; 44 words = 176 bytes):
39 --
40 -- Disassembly (address : encoding : instruction):
41 -- -- Setup --
42 --   0x0000: DE29 MOV.L @(0x29,PC),R14    R14 = 0x00001000
43 --   0x0002: 0009 NOP
44 -- -- Test 1: ADD Rm,Rn --
45 --   0x0004: E105 MOV #5,R1
46 --   0x0006: E203 MOV #3,R2
47 --   0x0008: 321C ADD R1,R2                R2 = 5+3 = 8
48 --   0x000A: 2E22 MOV.L R2,@R14            [0x1000] = 0x00000008
49 --   0x000C: 7E04 ADD #4,R14
50 -- -- Test 2: ADD #imm --
51 --   0x000E: E105 MOV #5,R1
52 --   0x0010: 71FD ADD #-3,R1                R1 = 5-3 = 2
53 --   0x0012: 2E12 MOV.L R1,@R14            [0x1004] = 0x00000002
54 --   0x0014: 7E04 ADD #4,R14
55 -- -- Test 3: SUB Rm,Rn --
56 --   0x0016: E10A MOV #10,R1
57 --   0x0018: E203 MOV #3,R2
58 --   0x001A: 3128 SUB R2,R1                R1 = 10-3 = 7
59 --   0x001C: 2E12 MOV.L R1,@R14            [0x1008] = 0x00000007
60 --   0x001E: 7E04 ADD #4,R14
61 -- -- Test 4: ADDC no carry --
62 --   0x0020: E101 MOV #1,R1
63 --   0x0022: 6117 NOT R1,R1                R1 = 0xFFFFFFFF
64 --   0x0024: E201 MOV #1,R2
65 --   0x0026: 312E ADDC R2,R1                R1 = 0xFFFFFFFF+1+T(0) = 0xFFFFFFFF, C=0

```

```

66 --      0x0028: 2E12  MOV.L R1,@R14      [0x100C] = 0xFFFFFFFF
67 --      0x002A: 7E04  ADD #4,R14
68 --  -- Test 5: ADDC with carry --
69 --      0x002C: E100  MOV #0,R1
70 --      0x002E: 6117  NOT R1,R1          R1 = 0xFFFFFFFF
71 --      0x0030: E201  MOV #1,R2
72 --      0x0032: 312E  ADDC R2,R1         R1 = 0xFFFFFFFF+1+T(0) = 0, C=1
73 --      0x0034: 2E12  MOV.L R1,@R14      [0x1010] = 0x00000000
74 --      0x0036: 7E04  ADD #4,R14
75 --  -- Test 6: MOVT (T=1 from carry) --
76 --      0x0038: 0329  MOVT R3          R3 = T = 1
77 --      0x003A: 2E32  MOV.L R3,@R14      [0x1014] = 0x00000001
78 --      0x003C: 7E04  ADD #4,R14
79 --  -- Test 7: ADDV overflow --
80 --      0x003E: D11B  MOV.L @(0x1B,PC),R1  R1 = 0x7FFFFFFF
81 --      0x0040: E201  MOV #1,R2
82 --      0x0042: 312F  ADDV R2,R1         R1 = 0x7FFFFFFF+1 = 0x80000000, V=1
83 --      0x0044: 2E12  MOV.L R1,@R14      [0x1018] = 0x80000000
84 --      0x0046: 7E04  ADD #4,R14
85 --  -- Test 8: NEG --
86 --      0x0048: E105  MOV #5,R1
87 --      0x004A: 621B  NEG R1,R2          R2 = -5 = 0xFFFFFFF
88 --      0x004C: 2E22  MOV.L R2,@R14      [0x101C] = 0xFFFFFFF
89 --      0x004E: 7E04  ADD #4,R14
90 --  -- Test 9: NEGC --
91 --      0x0050: 0008  CLRT              T = 0
92 --      0x0052: E101  MOV #1,R1
93 --      0x0054: 621A  NEGC R1,R2         R2 = 0-1-T(0) = 0xFFFFFFF
94 --      0x0056: 2E22  MOV.L R2,@R14      [0x1020] = 0xFFFFFFF
95 --      0x0058: 7E04  ADD #4,R14
96 --  -- Test 10: DT loop --
97 --      0x005A: E103  MOV #3,R1
98 --      0x005C: 4110  DT R1              R1 = R1-1, T = (R1==0)
99 --      0x005E: 8BFD  BF -1              loop to 0x005C if T=0
100 --     0x0060: 0329  MOVT R3              R3 = T = 1
101 --     0x0062: 2E32  MOV.L R3,@R14      [0x1024] = 0x00000001
102 --     0x0064: 7E04  ADD #4,R14
103 --  -- Test 11: CMP/EQ --
104 --     0x0066: E12A  MOV #42,R1
105 --     0x0068: E22A  MOV #42,R2
106 --     0x006A: 3120  CMP/EQ R2,R1        T = (42==42) = 1
107 --     0x006C: 0329  MOVT R3              R3 = 1
108 --     0x006E: 2E32  MOV.L R3,@R14      [0x1028] = 0x00000001
109 --     0x0070: 7E04  ADD #4,R14
110 --  -- Test 12: CMP/HS --
111 --     0x0072: E10A  MOV #10,R1
112 --     0x0074: E205  MOV #5,R2
113 --     0x0076: 3122  CMP/HS R2,R1        T = (10>=5 unsigned) = 1
114 --     0x0078: 0329  MOVT R3              R3 = 1
115 --     0x007A: 2E32  MOV.L R3,@R14      [0x102C] = 0x00000001
116 --     0x007C: 7E04  ADD #4,R14
117 --  -- Test 13: CMP/GT --
118 --     0x007E: E106  MOV #6,R1
119 --     0x0080: E205  MOV #5,R2
120 --     0x0082: 3127  CMP/GT R2,R1        T = (6>5 signed) = 1
121 --     0x0084: 0329  MOVT R3              R3 = 1
122 --     0x0086: 2E32  MOV.L R3,@R14      [0x1030] = 0x00000001
123 --     0x0088: 7E04  ADD #4,R14
124 --  -- Test 14: CMP/PZ --
125 --     0x008A: E100  MOV #0,R1
126 --     0x008C: 4111  CMP/PZ R1           T = (0==0) = 1
127 --     0x008E: 0329  MOVT R3              R3 = 1
128 --     0x0090: 2E32  MOV.L R3,@R14      [0x1034] = 0x00000001
129 --     0x0092: 7E04  ADD #4,R14
130 --  -- Test 15: SUBV underflow --
131 --     0x0094: D106  MOV.L @(0x06,PC),R1  R1 = 0x80000000
132 --     0x0096: E201  MOV #1,R2

```

```

133 --      0x0098: 312B  SUBV R2,R1          R1 = 0x80000000-1 = 0x7FFFFFFF, V=1
134 --      0x009A: 2E12  MOV.L R1,@R14      [0x1038] = 0x7FFFFFFF
135 --      0x009C: A000  BRA .               self-loop
136 --      0x009E: 0009  NOP                (delay slot)
137 --      0x00A0: 0009  NOP
138 --      0x00A2: 0009  NOP
139 --      -- Data constants --
140 --      0x00A4: 00001000
141 --      0x00A8: 7FFFFFFF
142 --      0x00AC: 80000000
143 -----
144
145 library ieee;
146 use ieee.std_logic_1164.all;
147 use ieee.numeric_std.all;
148 use std.textio.all;
149
150 entity TB2_Arithmetic is
151 end entity;
152
153 architecture sim of TB2_Arithmetic is
154
155     constant CLK_PERIOD : time      := 20 ns;
156     constant PROG_HEX   : string    := "tb2_program.hex";
157     constant MEM_WORDS  : integer   := 256;
158
159     signal clk          : std_logic := '0';
160     signal reset        : std_logic := '0';
161     signal loading      : std_logic := '1';
162
163     -- Shared bus - CPU, memory, and TB all connect here
164     signal AB : std_logic_vector(31 downto 0);
165     signal DB : std_logic_vector(31 downto 0);
166     signal RE0, RE1, RE2, RE3 : std_logic;
167     signal WE0, WE1, WE2, WE3 : std_logic;
168
169     -- TB-side bus signals
170     signal tb_AB : std_logic_vector(31 downto 0) := (others => '0');
171     signal tb_DB : std_logic_vector(31 downto 0) := (others => '0');
172     signal tb_RE : std_logic := '1';
173     signal tb_WE : std_logic := '1';
174
175 begin
176
177     clk <= not clk after CLK_PERIOD / 2;
178
179     -- TB bus mux: drives bus when loading='1', releases ('Z') when loading='0'
180     -- During loading='0' the CPU drives the bus instead (via its port map)
181     AB <= tb_AB when loading = '1' else (others => 'Z');
182     DB <= tb_DB when (loading = '1' and tb_WE = '0') else (others => 'Z');
183     RE0 <= tb_RE when loading = '1' else 'Z';
184     RE1 <= tb_RE when loading = '1' else 'Z';
185     RE2 <= tb_RE when loading = '1' else 'Z';
186     RE3 <= tb_RE when loading = '1' else 'Z';
187     WE0 <= tb_WE when loading = '1' else 'Z';
188     WE1 <= tb_WE when loading = '1' else 'Z';
189     WE2 <= tb_WE when loading = '1' else 'Z';
190     WE3 <= tb_WE when loading = '1' else 'Z';
191
192     -- CPU connects directly to the shared bus
193     dut : entity work.SH2_CPU
194     port map (
195         Reset => reset, NMI => '1', INT => '1', clock => clk,
196         AB => AB, DB => DB,
197         RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
198         WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3
199     );

```

```

200
201 mem : entity work.MEMORY32x32
202     generic map (
203         MEMSIZE      => MEM_WORDS,
204         START_ADDR0  => 16#00000000#,
205         START_ADDR1  => 16#00001000#,
206         START_ADDR2  => 16#10000000#,
207         START_ADDR3  => 16#20000000#
208     )
209     port map (
210         RE0 => RE0,  RE1 => RE1,  RE2 => RE2,  RE3 => RE3,
211         WE0 => WE0,  WE1 => WE1,  WE2 => WE2,  WE3 => WE3,
212         MemAB => AB,  MemDB => DB
213     );
214
215 stim : process
216     file      hex_file : text;
217     variable fstatus  : file_open_status;
218     variable l        : line;
219     variable word     : std_logic_vector(31 downto 0);
220     variable addr     : unsigned(31 downto 0);
221     variable pass_cnt : integer := 0;
222     variable fail_cnt : integer := 0;
223
224     procedure check(a : std_logic_vector(31 downto 0);
225                     exp : std_logic_vector(31 downto 0);
226                     tag : string) is
227     begin
228         tb_AB <= a;
229         tb_RE <= '0';
230         tb_WE <= '1';
231         wait for CLK_PERIOD;
232         if DB = exp then
233             pass_cnt := pass_cnt + 1;
234             report "PASS: " & tag severity note;
235         else
236             fail_cnt := fail_cnt + 1;
237             report "FAIL: " & tag
238                 & " got=0x" & to_hstring(DB)
239                 & " exp=0x" & to_hstring(exp) severity error;
240         end if;
241         tb_RE <= '1';
242         wait for CLK_PERIOD;
243     end procedure;
244
245 begin
246     -----
247     -- LOAD PHASE
248     -----
249     loading <= '1';
250     reset   <= '0';
251     tb_RE   <= '1';
252     tb_WE   <= '1';
253     wait for CLK_PERIOD * 2;
254
255     file_open(fstatus, hex_file, PROG_HEX, read_mode);
256     assert fstatus = open_ok
257         report "Cannot open " & PROG_HEX severity failure;
258
259     addr := (others => '0');
260     while not endfile(hex_file) loop
261         readline(hex_file, l);
262         if l'length > 0 then
263             hread(l, word);
264             tb_AB <= std_logic_vector(addr);
265             tb_DB <= word;
266             tb_WE <= '0';

```

```

267         wait for CLK_PERIOD;
268         tb_WE <= '1';
269         wait for CLK_PERIOD;
270         addr := addr + 4;
271     end if;
272 end loop;
273 file_close(hex_file);
274
275 -----
276 -- CPU PHASE
277 -----
278 wait until rising_edge(clk);
279 loading <= '0';
280 reset    <= '1';
281
282 wait for CLK_PERIOD * 2;
283
284 for i in 1 to 300 loop
285     wait until rising_edge(clk);
286 end loop;
287
288 -----
289 -- CHECK PHASE
290 -----
291 wait until rising_edge(clk);
292 reset    <= '0';
293 loading <= '1';
294 tb_RE    <= '1';
295 tb_WE    <= '1';
296 wait for CLK_PERIOD * 2;
297
298 check(x"00001000", x"00000008", "ADD Rm,Rn: 5+3=8");
299 check(x"00001004", x"00000002", "ADD #imm: 5+(-3)=2");
300 check(x"00001008", x"00000007", "SUB Rm,Rn: 10-3=7");
301 check(x"0000100C", x"FFFFFFFF", "ADDC no carry: 0xFFFFFFFFE+1");
302 check(x"00001010", x"00000000", "ADDC carry: result=0");
303 check(x"00001014", x"00000001", "ADDC carry: T=1 stored");
304 check(x"00001018", x"80000000", "ADDV overflow result");
305 check(x"0000101C", x"FFFFFFFB", "NEG 5 = 0xFFFFFFFB");
306 check(x"00001020", x"FFFFFFF", "NEGC 0-1-0 = 0xFFFFFFF");
307 check(x"00001024", x"00000001", "DT: T=1 after count=0");
308 check(x"00001028", x"00000001", "CMP/EQ 42==42: T=1");
309 check(x"0000102C", x"00000001", "CMP/HS 10>=5 unsigned: T=1");
310 check(x"00001030", x"00000001", "CMP/GT 6>5 signed: T=1");
311 check(x"00001034", x"00000001", "CMP/PZ 0>=0: T=1");
312 check(x"00001038", x"7FFFFFFF", "SUBV 0x80000000-1 underflow");
313
314 -----
315 -- SUMMARY
316 -----
317 report "TB2: PASSED=" & integer'image(pass_cnt)
318     & " FAILED=" & integer'image(fail_cnt) severity note;
319 if fail_cnt = 0 then
320     report "*** ALL TESTS PASSED ***" severity note;
321 else
322     assert false
323     report integer'image(fail_cnt) & " TEST(S) FAILED"
324     severity failure;
325 end if;
326 wait;
327 end process;
328
329 end architecture;
330

```